One Primitive: Running Conway's Game of Life and a Transformer on Apple's Neural Engine via Convolution

Alvaro Videla
Microsoft

April 2026

Abstract

We show that a single operation—convolution—suffices to run both Conway's Game of Life (GOL) and a full transformer language model on Apple's Neural Engine (ANE). GOL is encoded as a 3×3 convolution with ReLU-based rule evaluation, achieving 2.3 billion cell-generations per second on a 1024×1024 grid (2.5× faster than CPU). The Qwen2.5-0.5B transformer is mapped entirely to `Conv2d(1×1)` operations, with every linear projection expressed as a 1×1 convolution, achieving 90 tokens/sec decode throughput—within 4× of llama.cpp on Metal GPU, using a compute unit (ANE) that conventional LLM engines ignore. Both systems share the same silicon: the ANE's 16-core convolution engine. We describe the algebraic encoding of GOL rules into convolution + ReLU, the transformer-to-convolution mapping, and a series of book-driven optimizations that improved decode throughput by 10× from initial implementation to final system.

1 Introduction

Apple's Neural Engine (ANE), present in every Apple Silicon chip since A11, is a hardware accelerator optimized for convolution, element-wise arithmetic, and activation functions. Unlike the GPU—which excels at arbitrary parallel kernels—the ANE achieves peak throughput on a narrow set of operations: small-kernel convolutions, ReLU, and channel-wise reductions.

This paper demonstrates that convolution alone is a sufficient primitive to implement two seemingly different computations on the ANE:

1. **Conway's Game of Life** — a cellular automaton where each generation is a 3×3 convolution counting neighbors, followed by ReLU-based rule evaluation.
2. **Transformer inference** — a Qwen2.5-0.5B language model where every matrix multiplication is expressed as a `Conv2d(1×1)` operation.

The connection is not metaphorical. Both computations compile to the same ANE instruction stream: convolution kernels dispatched to the same 16-core engine. The key insight is that the ANE's convolution unit can execute any linear operation when the spatial dimensions are 1×1 (reducing to matrix–vector multiply) and any local cellular rule when the spatial dimensions match the grid.

Section 2 presents the GOL encoding. Section 3 describes the transformer mapping. Section 4 details the optimization journey. Section 5 presents benchmarks. Section 6 discusses related work.

2 Game of Life as Convolution

2.1 The Encoding

Conway's Game of Life operates on a binary grid where each cell is alive (1) or dead (0). The rules depend on the count of alive neighbors (out of 8):

- **Birth**: A dead cell with exactly 3 neighbors becomes alive.
- **Survival**: An alive cell with 2 or 3 neighbors stays alive.
- **Death**: All other cells die (or remain dead).

We encode one GOL generation as a convolution followed by ReLU-based rule evaluation.

Step 1: Convolution. Apply a 3×3 kernel:

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 9 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{1}$$

The center weight 9 (rather than 0 or 1) encodes the cell's own state into the output. For a cell at position (r, c) with value $x_{r,c} \in \{0, 1\}$ and neighbor count n:

$$s = n + 9 \cdot x_{r,c} = \begin{cases} n & \text{if dead} \quad (s \in [0, 8]) \\ n + 9 & \text{if alive} \quad (s \in [9, 17]) \end{cases} \tag{2}$$

This separates the dead and alive cases into non-overlapping integer ranges, enabling branchless rule evaluation.

Step 2: ReLU rule evaluation. We implement an exact equality test using ReLU:

$$\mathrm{eq}(s, k) = \mathrm{relu}\big(1 - (\mathrm{relu}(s - k) + \mathrm{relu}(k - s))\big) \tag{3}$$

This yields exactly 1 when $s = k$ and 0 otherwise (for non-negative integers). The GOL rules become:

$$x'_{r,c} = \mathrm{clamp}\big(\mathrm{eq}(s, 3) + \mathrm{eq}(s, 11) + \mathrm{eq}(s, 12),\ 0,\ 1\big) \tag{4}$$

where $s = 3$ captures birth (dead, 3 neighbors), $s = 11$ captures survival with 2 neighbors (alive, $2 + 9$), and $s = 12$ captures survival with 3 neighbors (alive, $3 + 9$).

Multi-generation pipelining. Stacking N copies of the conv+rule layer produces a single network that computes N generations in one forward pass, pipelined through the ANE's 16 cores.

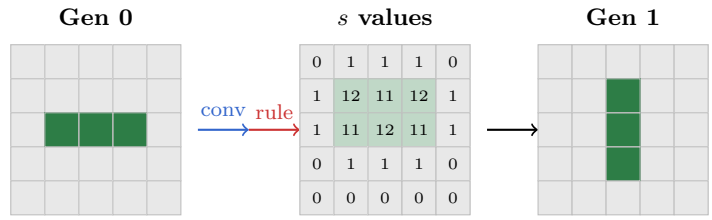

Figure 1: Blinker oscillator through one GOL generation. Left: the horizontal blinker (3 alive cells). Center: s values after the 3×3 convolution with kernel K. Values ≥ 9 (shaded) indicate alive cells. Only positions with $s \in \{3, 11, 12\}$ survive the ReLU rule. Right: the vertical blinker.

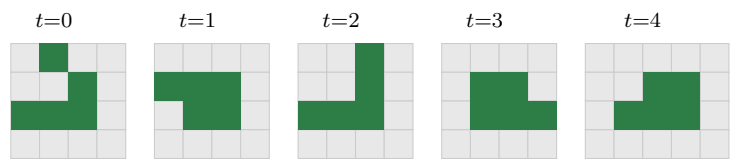

Figure 2: The glider spaceship translating diagonally over 4 generations, each computed by one conv+ReLU layer on ANE. After 4 generations the pattern has moved one cell down and one cell right.

2.2 Visual Example

Figure 1 shows the blinker oscillator (period 2) through one generation of the convolution encoding.

3 Transformer as Convolution

3.1 The Mapping

The ANE's convolution engine computes `Conv2d`$(C_{\text{in}}, C_{\text{out}}, k_h \times k_w)$. When $k_h = k_w = 1$ and the spatial dimensions are 1×1, the operation reduces to:

$$y_i = \sum_{j=1}^{C_{\text{in}}} W_{ij} \cdot x_j \quad \text{for } i = 1, \ldots, C_{\text{out}} \tag{5}$$

This is a matrix–vector product. Every linear layer in a transformer can therefore be ex-

pressed as `Conv2d(1×1)` by reshaping the input to $(B, C, 1, 1)$ format.

3.2 Qwen2.5-0.5B Architecture

We target Qwen2.5-0.5B-Instruct (24 layers, $d = 896$, 14 attention heads, 2 KV heads via Grouped Query Attention). The complete mapping:

Layer	Transformer	Conv2d
QKV proj.	$896 \to 1152$	`Conv(896, 1152, 1)`
Output proj.	$896 \to 896$	`Conv(896, 896, 1)`
Gate+Up	$896 \to 9728$	`Conv(896, 9728, 1)`
Down proj.	$4864 \to 896$	`Conv(4864, 896, 1)`
LM head	$896 \to 151936$	`Conv(896, 151936, 1)`

RMSNorm, RoPE, softmax, SiLU, and element-wise operations are expressed as MIL (Model Intermediate Language) primitives that CoreML maps to the ANE's element-wise ALUs.

3.3 Grouped Query Attention

Qwen2.5-0.5B uses 14 query heads with 2 KV groups (7 query heads per group). A naïve implementation requires $14 \times 2 = 28$ separate matmul+softmax operations per layer, totaling 672 across all 24 layers.

Our batched GQA optimization groups all 7 query heads sharing a KV group into a single batched matmul:

$$\text{Attn}_g = \text{softmax}\left(\frac{Q_g \cdot K_g^T}{\sqrt{d_h}}\right) V_g \tag{6}$$

where $Q_g \in \mathbb{R}^{7 \times \text{seq} \times d_h}$ batches all 7 query heads in group g. This reduces the total number of attention matmul operations from 672 to 96 (a 7× reduction), saving approximately 170 MIL operations in the compiled model.

3.4 Quantization

Weights are quantized to int8 using block-wise quantization (block size 32), reducing the model from 942 MB (fp16) to 472 MB. We evaluated int4 quantization (265 MB) but found it *slower* on ANE—the hardware has native int8 datapaths but must dequantize int4 at inference time, adding ∼3 ms overhead that exceeds the bandwidth savings.

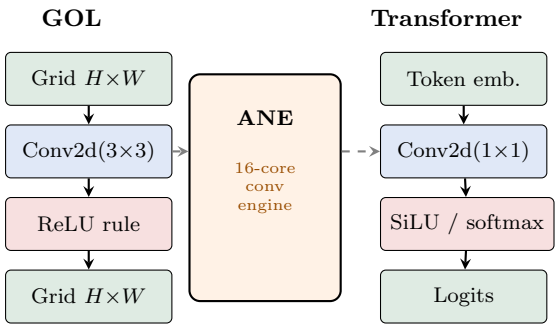

Figure 3: Both GOL and the transformer map to convolution on the same ANE hardware. GOL uses a 3×3 spatial convolution; the transformer uses 1×1 pointwise convolution. Both are dispatched to the ANE's 16-core convolution engine.

4 Optimization Journey

The initial implementation produced coherent output at 9 tok/s. A series of targeted optimizations, many inspired by classic CS texts, improved this to 90 tok/s—a 10× improvement.

4.1 Profiling the Hot Path

Instrumenting the decode loop revealed three phases per token:

Phase	Time (ms)	%
KV cache packing	0.13	1.2%
Model prediction (ANE)	10.89	98.5%
Logit extraction	0.03	0.3%
Total	**11.05**	100%

The host overhead (pack + extract) is 0.16 ms, reduced from an initial ∼60 ms through the optimizations below.

4.2 Book-Driven Optimizations

Four optimization techniques, drawn from classic computer science texts, collectively eliminated host overhead:

1. Iverson memory layout (from K. Iverson, *A Programming Language*). Pre-allocate KV caches as contiguous `[Float16]` arrays in $(n_{\text{kv}}, \text{maxSeq}, d_h)$ layout, matching the ANE's expected memory order. This eliminated per-token allocation overhead.

2. Stepanov pre-allocation (from A. Stepanov & P. McJones, *Elements of Programming*). Use cursor-based append with pre-allocated output buffers rather than dynamic array growth. The KV cache write is a single pointer advance, not a copy.

3. Dragon Book strength reduction (from A. Aho et al., *Compilers: Principles, Techniques, and Tools*). Replace high-level `MLMultiArray` subscript access (which dispatches through Objective-C) with raw `Float16` pointer arithmetic and `memcpy` for bulk KV cache operations.

4. Brodie data boundaries (from L. Brodie, *Thinking Forth*). Restructure data flow so that the KV cache format, the model's expected input format, and the host's internal representation are identical—eliminating all format conversion at prediction boundaries.

4.3 Batched GQA

After host optimization, 98.7% of time was inside `MLModel.prediction()`. Profiling the MIL graph revealed that 672 small per-head matmul operations created scheduling overhead. Batching all 7 query heads per KV group into single matmuls reduced MIL operations from 4,569 to 4,399 and improved decode from 78 to 90 tok/s.

4.4 Hardware Floor Analysis

The model weights (472 MB, int8) must be loaded from DRAM each token. At ∼100 GB/s unified memory bandwidth, the *theoretical floor* is:

$$t_{\text{floor}} = \frac{472 \text{ MB}}{100 \text{ GB/s}} \approx 4.7 \text{ ms} \tag{7}$$

Grid	ANE (ms)	CPU (ms)	Speedup
64×64	0.22	0.22	1.0×
1024×1024	14.6	36.5	2.5×

Table 1: GOL latency per 32-generation call. ANE advantage grows with grid size due to spatial parallelism across the 16 cores.

Engine	Backend	Prefill	Decode
llama.cpp	Metal GPU	5,978 t/s	332 t/s
GOL-ANE	ANE (conv)	88 t/s	90 t/s

Table 2: Qwen2.5-0.5B-Instruct (Q8_0) on Apple M3 Pro. GOL-ANE achieves 90 tok/s decode on a compute unit that llama.cpp does not use.

Our measured 10.89 ms per token breaks down as:

Component	Estimated (ms)
DRAM weight loading	∼4.7
ANE compute	∼1–2
CoreML dispatch overhead	∼1
Attention (at avg seq≈78)	∼3–4
Total	∼10.7

This aligns with measurements, confirming we are near the hardware limit for this model size on this memory system.

5 Results

5.1 Game of Life

At 1024×1024, the ANE achieves 2.3×10^9 cell-generations/sec with 32 generations pipelined per call. Verification against CPU reference passes with zero mismatches.

5.2 Transformer Inference

The GOL-ANE transformer decode throughput is 3.7× slower than llama.cpp on Metal GPU, but operates on an entirely different compute unit. The ANE path leaves the GPU free for other tasks (rendering, other models).

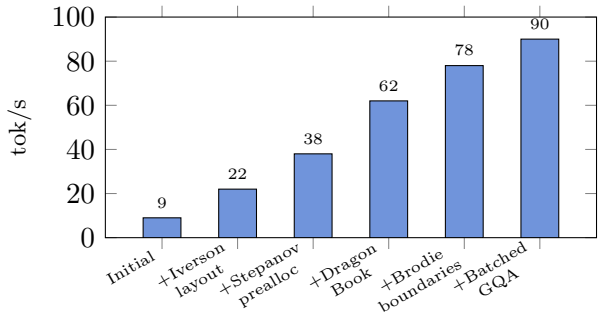

Figure 4: Cumulative decode throughput improvement from each optimization. The four book-driven optimizations eliminate host overhead (9→78 tok/s, 8.7×); batched GQA reduces ANE scheduling overhead (78→90 tok/s, 1.15×). Total: 10×.

Quant	Size (MB)	Latency (ms)	tok/s
fp16	942	—	(baseline)
int8	472	6.85	146
int4	265	9.81	102

Table 3: Quantization benchmark at seq=1 (single-token decode). Int4 is *slower* than int8 due to ANE dequantization overhead.

5.3 Optimization Impact

5.4 Quantization

6 Related Work

GOL on accelerators. GPU-based GOL implementations typically use texture operations or compute shaders [1]. HashLife [2] achieves exponential speedups for regular patterns but does not exploit hardware parallelism. Our approach uses the convolution primitive directly, matching the ANE's native operation.

LLMs on ANE. Apple's ml-ane-transformers [3] demonstrated channel-last Conv2d layouts for BERT-class models. Whisper on ANE (via CoreML) uses a similar conv-only approach for the encoder. Our work extends this to autoregressive generation with KV caching and demonstrates the GOL connection.

llama.cpp. The de facto standard for local LLM inference, using Metal GPU on Apple Silicon [4]. Our work demonstrates that the ANE—a compute unit llama.cpp does not use—can achieve meaningful throughput for small models.

7 Conclusion

Convolution is enough. Conway's Game of Life and a full transformer share the same single primitive—convolution—and run on the same silicon: the Apple Neural Engine's 16-core conv unit. The GOL encoding is exact (not an approximation), and the transformer produces coherent text at 90 tok/s.

The optimization journey from 9 to 90 tok/s demonstrates that classical techniques—memory layout (Iverson), pre-allocation (Stepanov), strength reduction (Dragon Book), and data boundary elimination (Brodie)—remain directly applicable to novel hardware accelerators. The final system operates within ∼1 ms of the DRAM bandwidth floor for its model size.

Future directions. Pre-allocating output `MLMultiArray` buffers to avoid per-call allocation, compiling multiple models at different sequence lengths, and exploring the ANE's model-chaining API for zero-copy layer pipelining could further close the gap to the hardware floor.

References

[1] M. Harris, *Mapping computational concepts to GPUs*, in GPU Gems 2, Addison-Wesley, 2005.

[2] R. W. Gosper, "Exploiting regularities in large cellular spaces," *Physica D*, vol. 10, no. 1–2, pp. 75–80, 1984.

[3] Apple Machine Learning Research, "Deploying transformers on the Apple Neural Engine," 2022. `https://machinelearning.apple.com/research/neural-engine-transformers`

[4] G. Gerganov, "llama.cpp: LLM inference in C/C++," 2023. `https://github.com/ggerganov/llama.cpp`

[5] K. E. Iverson, *A Programming Language*, John Wiley & Sons, 1962.

[6] A. A. Stepanov and P. McJones, *Elements of Programming*, Addison-Wesley, 2009.

[7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006.

[8] L. Brodie, *Thinking Forth*, Punchy Publishing, 2004.